Two-Axis Embedded Light Tracking System

with FSM-Based PID Servo Control

Personal Project Documentation

Embedded Systems • Control Theory • Renewable Energy

Contents

Abstract 3
1. Motivation and Objectives 3
2. System Overview 3
2.1 Component Summary 4
3. Hardware Design 5
3.1 Sensor Front-End 5
3.2 Actuation 6
3.3 Power Distribution 6
4. Software Architecture 6
4.1 State Definitions 6
4.2 State Transition Logic 7
5. Control Algorithm 7
5.1 Error Computation 7
5.2 PID Implementation 8
5.3 Stability Measures 8
6. Testing Procedure 8
6.1 Test 1 – Sensor Verification 9
6.2 Test 2 – Error Sign Verification 9
6.3 Test 3 – Open-Loop Servo Direction 9
6.4 Test 4 – Closed-Loop Tracking 9
6.5 Test 5 – Extreme Position Test 9
7. Problems Encountered and Solutions 10
8. Final Tuned Parameters/Results 11
9. Future Improvements 12
10. Conclusion 12

Abstract

This project presents a two-axis solar tracking system that automatically rotates a sensor platform to align with the direction of strongest incident light. Four light-dependent resistors (LDRs) are arranged into four quadrants and separated by physical shadow dividers, allowing the microcontroller to compute horizontal and vertical light-intensity gradients. An Arduino Nano processes the four LDR signals, derives error values along each axis, and drives two servo motors using a finite state machine (FSM) coupled with a PID controller. The system incorporates analog filtering, deadband logic, search/track/hold states, and mechanical command limits to prevent unstable oscillation and protect the actuators.

1. Motivation and Objectives

Modern industry depends on a finite supply of conventional energy, and renewable sources are becoming increasingly critical to meet large-scale energy demand. Solar energy in particular benefits significantly from active panel orientation: a panel that continuously faces the strongest light source captures more energy than a fixed-orientation panel.

This project mimics the heliotropic behaviour of a sunflower, which gradually reorients itself toward the sun throughout the day. The same principle can be extended to commercial solar panel positioning systems, where mechanical tracking improves overall energy yield. The objectives of this project are:

- Design a closed-loop two-axis tracker capable of locating and following the strongest light source in its field of view.
- Implement an FSM-based control architecture with clearly defined operating modes (Off, Initialize, Search, Sleep, Track, Hold).
- Apply PID control with deadband and saturation limits to achieve stable, oscillation-free tracking.
- Validate the system experimentally with structured open-loop and closed-loop tests.

2. System Overview

The system consists of three functional layers: a sensing front-end, an embedded processing unit, and a mechanical actuation stage. Light intensity is sensed by four LDRs arranged in a 2×2 grid behind a shadow divider. Each LDR forms a voltage divider, whose output is filtered by an RC low-pass network to suppress sensor noise. The filtered signals are read by the Arduino Nano, which executes the FSM and PID control loop. The resulting control signals drive two servo motors that actuate the pan and tilt axes of the platform. A separate regulated 5 V supply powers the servos to isolate their current spikes from the microcontroller rail.

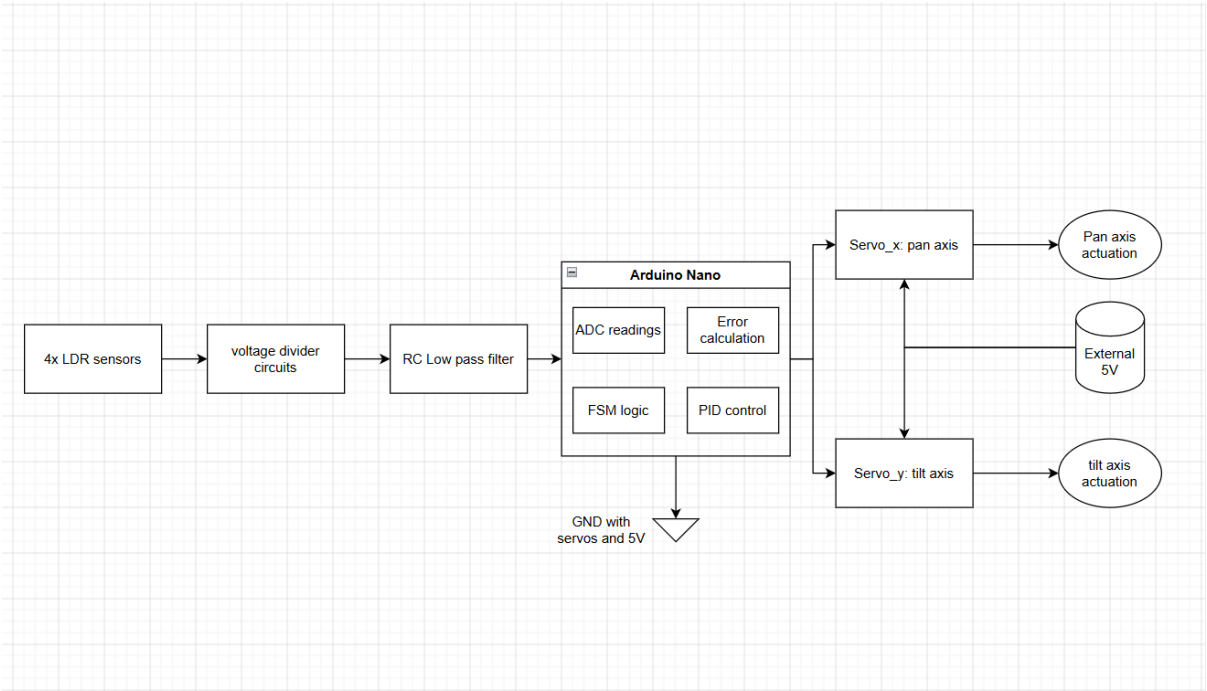

Figure 1 system architecture/overview

2.1 Component Summary

Component	Purpose
4× LDR sensors	Detect light intensity differences between the left/right and top/bottom halves of the platform.
Voltage dividers	Convert light-dependent resistance into a measurable analog voltage.
RC low-pass filters	Reduce high-frequency sensor noise before sampling.
Arduino Nano	Reads sensor values, runs the FSM, computes axis errors, and outputs PID control signals.
Servo motors (×2)	Provide pan (yaw) and tilt (pitch) actuation of the sensor platform.
External 5 V supply	Powers the servos independently of the microcontroller rail.

3. Hardware Design

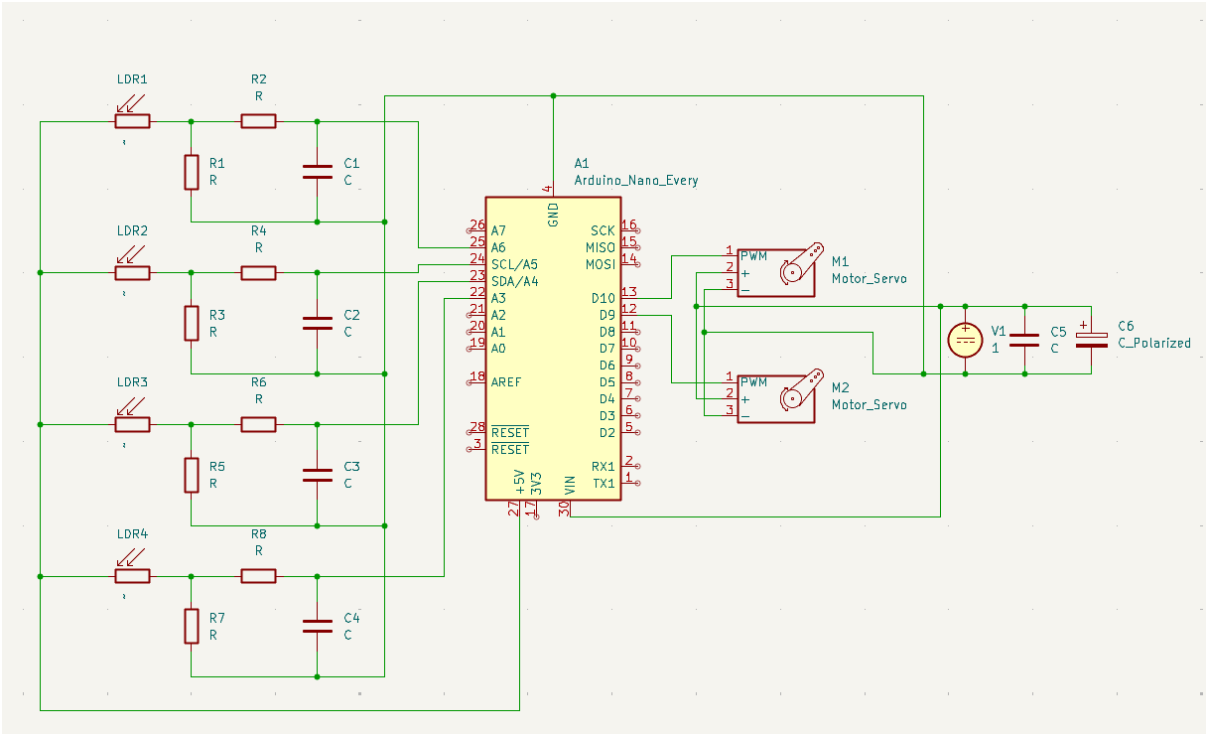

Figure 2 Hardware schematic

3.1 Sensor Front-End

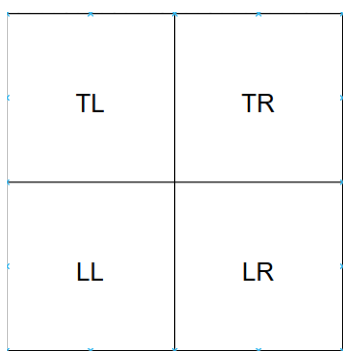

Figure 3 LDR layout

The four LDRs are arranged in the following quadrant layout when viewed from the front of the platform:

Top-Left (TL)	Top-Right (TR)
Lower-Left (LL)	Lower-Right (LR)

Each LDR is paired with a fixed resistor to form a voltage divider, and a parallel capacitor implements an RC low-pass filter. The filtered output is fed to one of the Arduino's analog input pins:

```
int TL = analogRead(A6);
int TR = analogRead(A3);
int LL = analogRead(A5);
int LR = analogRead(A4);
```

3.2 Actuation

- Servo 1 controls horizontal (pan / yaw) movement.
- Servo 2 controls vertical (tilt / pitch) movement.
- Servo commands are issued via writeMicroseconds() in standard PWM-style pulses.
- The neutral position is approximately 1500 µs.
- Travel is restricted in software to roughly 550–2450 µs to prevent mechanical stress and lock-up.

3.3 Power Distribution

The servos are not powered from the Arduino Nano's regulated 5 V output. Servo current spikes can cause voltage sags that reset the microcontroller or trigger erratic behaviour. Instead, a separate regulated 5 V supply powers the servos, and a common ground is shared between the Arduino and the servo supply to ensure a consistent reference.

Two decoupling capacitors are placed across the servo supply rail to absorb transient current spikes:

- 0.1 µF ceramic capacitor for high-frequency noise.
- 10 µF electrolytic capacitor for bulk transient suppression.

4. Software Architecture

The control software is organised as a finite state machine. Each state represents a distinct operating mode of the tracker, with clearly defined entry conditions, behaviour, and exit transitions. This separation makes the system easier to reason about and easier to debug.

4.1 State Definitions

State	Purpose
Off	System is disabled. No servo commands are issued.
Initialize	Centres both servos and resets PID state variables (integrator, differentiator, previous error).
Search	Evaluates whether the measured light intensity difference is significant enough to begin tracking.
Sleep	Idle state entered when no significant light gradient is detected; waits for conditions to change.

State	Purpose
Track	Actively follows the strongest light source using closed-loop PID control.
Hold	Locks the servo positions when the error falls below the deadband, ensuring the platform remains aligned without micro-corrections.

4.2 State Transition Logic

The system always starts in the Off state. A digital input pin connected to a physical switch determines the on/off condition. When the switch is set to on, the system transitions to the Initialize state, which activates the servos by centring them and clearing the PID gain accumulators (integrator, differentiator, and previous error).

After initialisation, control passes to the Search state. Here the FSM evaluates whether the measured light-intensity difference exceeds a configured threshold. If the threshold is exceeded, the system transitions into the Track state and begins continuously following the strongest light source. If the threshold is not exceeded, the system enters the Sleep state and remains idle until the light gradient becomes significant again, at which point it returns to Search.

During Track, if the absolute error falls below the deadband, the FSM transitions into the Hold state. In Hold the servos are commanded to remain stationary, ensuring the platform remains aligned with the strongest light source without unnecessary micro-corrections. If the error subsequently grows beyond the deadband again, the system returns to Track.

5. Control Algorithm

5.1 Error Computation

Two scalar error signals are derived from the four LDR readings, one per axis. The horizontal (pan) error is the difference between the right- and left-hand pairs of LDRs, and the vertical (tilt) error is the difference between the upper and lower pairs:

```
float error_x(int TR, int LR, int TL, int LL) {
    // bottom motor = left/right yaw
    // positive x_error means right side is brighter
    int left_side  = TL + LL;
    int right_side = TR + LR;
    return float(right_side - left_side);
}

float error_y(int TR, int LR, int TL, int LL) {
    // top motor = up/down pitch
    // positive y_error means top side is brighter
```

```
    int top_side   = TL + TR;
    int lower_side = LL + LR;
    return float(top_side - lower_side);
}
```

A positive x_error indicates that the right-hand LDRs receive more light than the left-hand LDRs. The sign of the servo response was determined experimentally so that a positive error rotates the platform toward the stronger light source. The same convention applies to the vertical axis.

5.2 PID Implementation

The PID controller is implemented in discrete time. The integrator term accumulates error scaled by the loop period, and the differentiator term divides the error change by the same period. millis() is used to obtain the elapsed time since the previous iteration:

```
x_integrator = x_integratorPrev + x_error * (millis() - t0) * 0.001;
x_integrator = constrain(x_integrator, -500, 500);

y_integrator = y_integratorPrev + y_error * (millis() - t0) * 0.001;
y_integrator = constrain(y_integrator, -500, 500);

x_differentiator = (x_error - x_errorPrev) / ((millis() - t0) * 0.001);
y_differentiator = (y_error - y_errorPrev) / ((millis() - t0) * 0.001);

float ux = kp * x_error + kd * x_differentiator + ki * x_integrator;
float uy = kp * y_error + kd * y_differentiator + ki * y_integrator;
```

The integrator is constrained to the range [−500, 500] to prevent integral wind-up during prolonged saturation. The combined control outputs ux and uy are then mapped to servo pulse widths and applied to the pan and tilt actuators respectively.

5.3 Stability Measures

- A deadband around zero error suppresses constant small servo corrections caused by sensor noise, reducing jitter and mechanical wear.
- Servo command limits clip the output pulse width to a safe mechanical range, preventing the actuators from being driven into their hard stops.
- Integrator saturation limits prevent wind-up when the system cannot immediately reduce a large error, avoiding overshoot when the error eventually changes sign.

6. Testing Procedure

The system was validated through a sequence of structured tests, progressing from open-loop sensor verification to full closed-loop behaviour.

6.1 Test 1 – Sensor Verification

Purpose: confirm that each LDR responds correctly and is mapped to the expected analog input.

Method: shine a focused light source on each of the TL, TR, LL, and LR sensors in turn and print the ADC values to the serial monitor. Verify that the illuminated sensor's value increases or decreases as expected for the chosen voltage divider orientation.

6.2 Test 2 – Error Sign Verification

Purpose: confirm that the computed error signals have the correct sign with respect to the physical light direction.

Light position	Expected error
Right side	x_error becomes strongly positive (or negative, depending on convention)
Left side	Opposite sign to right-side case
Top side	y_error changes one way
Bottom side	Opposite sign to top-side case

6.3 Test 3 – Open-Loop Servo Direction

Purpose: confirm the control sign by manually commanding servo positions.

- Manually command posX += 100 and observe whether the servo turns right or left.
- Manually command posY += 100 and observe whether the servo turns up or down.
- Adjust the sign convention in software if the actual rotation does not match the intended direction.

6.4 Test 4 – Closed-Loop Tracking

Purpose: validate full closed-loop behaviour.

- Place a flashlight at different angles relative to the platform.
- Observe whether the platform converges toward the light source.
- Record whether it oscillates, saturates, or holds correctly once aligned.

6.5 Test 5 – Extreme Position Test

Purpose: verify that mechanical lock-up cannot occur.

- Move the light source far to the left, right, top, and bottom of the platform's field of view.
- Confirm that the servo command remains within the configured safe pulse-width limits in every case.

7. Problems Encountered and Solutions

The following issues were observed during development and tuning, along with their root causes and the corrective actions taken:

Problem	Cause	Solution
Servo barely moved	Kp set too low.	Increased Kp to 0.5 that produced visible tracking action.
Servo oscillation	Kp set too high.	decreased Kp to 0.1 until oscillation was no longer significant while keeping response time acceptable.
Tracking failed on the test bench	LDR orientations were incorrect; errors were unintentionally being computed across diagonals.	Tested each LDR individually and re-mapped the orientations to match the physical layout. (See hardware design section)
Oscillation reappeared after raising Kd	Higher Kd amplifies sensor noise into the control output.	Reduced Kd to 0.011 until the noise-driven response disappeared.
Tracker did not respond as expected	Deadband was set too wide and was masking real errors.	Reduced the deadband to 50 until the platform tracked the light while still ignoring noise-level fluctuations.

8. Final Tuned Parameters/Results

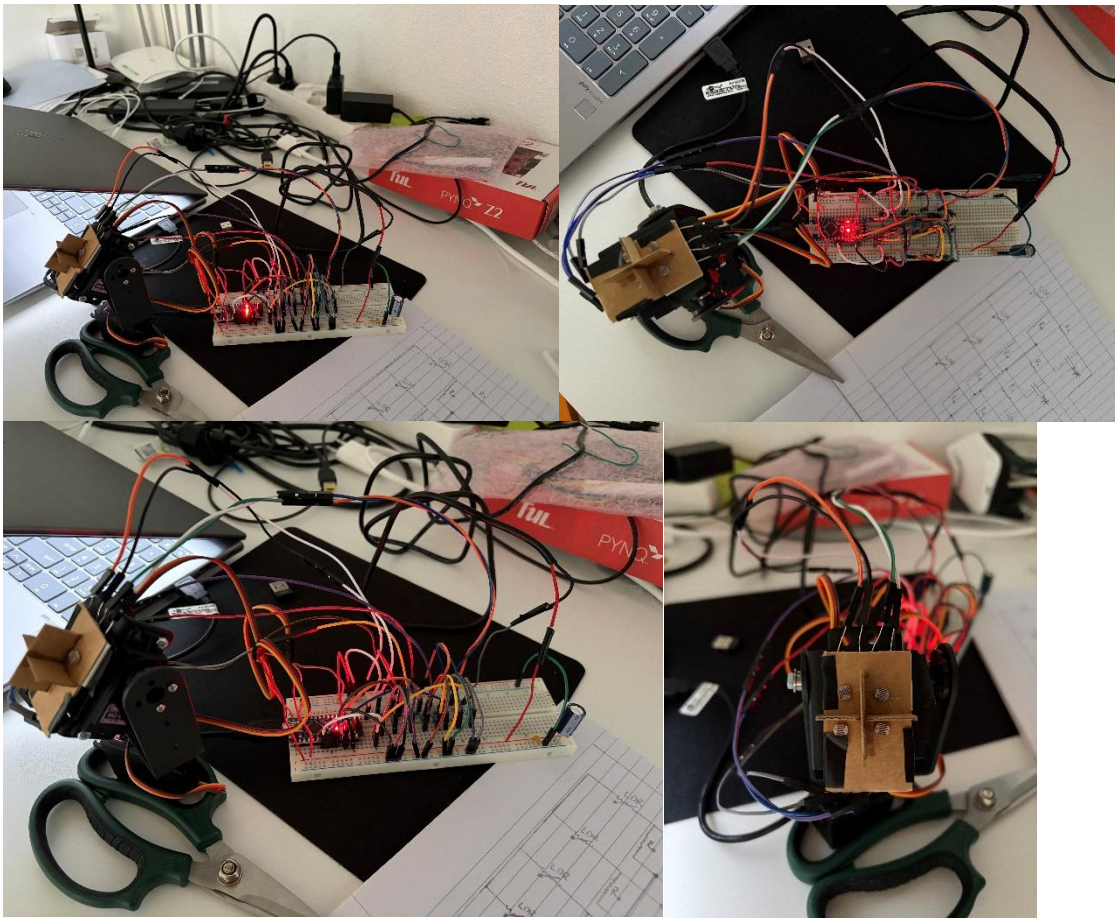

Figure 4 Final design pictures

https://drive.google.com/file/d/1waw5Laf0Yg75xCkR131URpU9xYq5q6HJ/view?usp=sharing

After iterative testing, the following parameter set produced reliable tracking behaviour with no sustained oscillation:

Parameter	Final value
Kp (proportional gain)	0.1
Kd (derivative gain)	0.011
Ki (integral gain)	1
Tracking threshold	35
Deadband	50
Servo minimum pulse	550 µs

Parameter	Final value
Servo maximum pulse	2450 µs
Search timeout	5 s

9. Future Improvements

The prototype tracked reliably within the central field of view, but tracking accuracy decreased at larger light angles. Serial debugging showed that the servo commands sometimes remained near the neutral 1500 µs position, indicating that the LDR error signal was too weak at those angles. This suggests that the current sensor geometry does not always produce a strong directional gradient. A taller and more rigid shadow-divider structure is required for improved full-range tracking.
Several refinements would extend the capability and robustness of the system:

- Mount the LDRs directly on the moving platform with enhanced physical shadow dividers for cleaner gradient measurement.
- Replace the breadboard wiring with a custom PCB to improve reliability and reduce noise.
- Add over-current protection and improved power distribution for the servo supply.
- Integrate a real solar panel and measure the generated electrical power as a function of tracker performance.
- Add time-based sun position prediction using date, time, and geographic location as a feed-forward term to complement the closed-loop tracker.
- Add data logging via SD card or serial plotter for offline analysis.
- Tune the PID gains systematically using step-response testing rather than purely empirical adjustment.
- Replace the LDRs with photodiodes or digital lux sensors for higher accuracy and a wider dynamic range.
- Add a manual override mode for diagnostics and demonstration.

10. Conclusion

This project demonstrates a working two-axis light tracking system built around an Arduino Nano, four LDR sensors, and two servo motors. The combination of an FSM-based supervisory layer with PID inner-loop control produced reliable tracking behaviour while remaining easy to reason about and debug. Stability was achieved through a combination of analogue filtering, integrator clamping, deadband logic, and software-imposed servo limits. The final tuned parameters yielded smooth convergence toward the strongest light source within the central operating range, without sustained oscillation. Testing also revealed that tracking accuracy decreases at larger light angles because the current LDR geometry produces a weaker directional error signal in those conditions. The architecture is directly transferable to larger-scale solar panel positioning systems, where the same control principles can be applied with more capable sensors and mechanically robust actuators.